

Individual Task 1: Part 1

HEET CHANCHAD (S4218211), RMIT University, Australia

This report examines the role of Machine Learning Engineer in Trust and Safety Engineering, using a live graduate advertisement at TikTok as its anchor. The professional problem the role exists to solve is triage under a fixed review budget: a platform receives far more user-generated content each day than any moderation team can read, so a classifier must rank items by the probability of violation and spend scarce human attention where it changes an outcome. Two public corpora of crowd-labelled user text are used to study that problem — 24,783 annotated tweets and 159,686 annotated Wikipedia Talk comments — and two machine learning algorithms, a linear support vector machine and a feed-forward neural network, are compared under stratified cross validation. Because the removal-worthy class covers only 5.8% and 9.6% of the two corpora respectively, the evaluation deliberately avoids accuracy in favour of precision, recall, area under the precision-recall curve, and precision over the slice of the ranked stream a fixed review team could actually process. The analysis finds that the two data sources are complementary in what they teach about model choice but directly contradictory in what they teach about labels, and that this contradiction is invisible to any single-dataset evaluation.

Overleaf project URL: <https://www.overleaf.com/read/swdgzknjst#264b95>

1 Data Science Role

1.1 Job Description and Role Identification

The role selected is **Machine Learning Engineer Graduate (Trust and Safety Engineering) – 2027 Start** at TikTok, based in Sydney, New South Wales, and open to international applicants. Applications close 30 August 2026. The full advertisement is reproduced in Appendix A.

The advertisement places the engineer inside TikTok’s Trust & Safety team, which describes itself as having “invested heavily in human and machine-based moderation to remove harmful content quickly and often before it reaches our general community”. The listed responsibilities are explicitly build-oriented: to “develop and build up highly-scalable classifiers, tools, models and algorithms leveraging cutting-edge machine learning, computer vision and data mining technologies”, to “improve our trust and safety strategy and work on model iterations”, and to “develop and implement innovative machine learning algorithms to manage business risks”. Minimum qualifications are a Master’s or PhD in computer science or a related engineering field, solid knowledge of at least one of machine learning, pattern recognition, natural language processing, data mining or computer vision, and a firm understanding of data structures and algorithms.

The professional problem behind the role is one of *scale under a fixed review budget*. A platform of TikTok’s size receives far more user-generated content each day than any moderation team can read, yet removing content automatically and without recourse is unacceptable to creators and regulators alike. Neither exhaustive human review nor fully autonomous removal is viable. The engineering problem is therefore triage: rank incoming items by the probability that they violate policy, so that a finite number of human moderators spend their attention where it changes an outcome. This is the problem the remainder of this report addresses, and it is what makes the role a machine learning *engineering* role rather than an analytics one — the deliverable is a ranked queue running in production, monitored and re-trained, not a slide deck.

Job listing URL: <https://au.gradconnection.com/employers/tiktok/jobs/tiktok-machine-learning-engineer-graduate-trust-and-safety-engineering-2027-start-5/>

1.2 Industry Context

Short-form video platforms cannot human-review millions of daily uploads. The trust and safety field uses data science to detect hate speech, harassment, spam and coordinated abuse, to rank what scarce moderators see first, and to monitor whether enforcement stays consistent across languages, dialects and regions.

[Word count: 45]

1.3 Role Focus

Prediction, delivered as production software. The advertisement asks the engineer to “develop and build up highly-scalable classifiers, tools, models and algorithms” and to “work on model iterations” to “manage business risks”. Insight generation is secondary; the deliverable is a deployed classifier that ranks content for moderator review.

[Word count: 47]

1.4 Value Creation

Automated triage lets a fixed moderation team cover an expanding content stream, so harmful material is removed before it spreads. Better ranking also protects creators from wrongful removal and advertisers from unsafe placement, while producing the enforcement evidence regulators such as Australia’s eSafety Commissioner require [6].

[Word count: 46]

1.5 Values and Culture Alignment

The team frames its mission as keeping people “safe and empowered to create”, which matches how I want to use machine learning: reducing harm rather than maximising engagement. I value measured claims over confident ones, and this role’s emphasis on experiments, validation and iteration rewards that habit.

[Word count: 47]

1.6 Diversity and Inclusion

Moderation models inherit their annotators’ judgements. The Wikipedia corpus used here ships annotator demographics precisely because who labels the data shapes what counts as toxic [13]. Diverse engineering teams are likelier to disaggregate error rates by dialect and community, catching the over-flagging that silences minority speakers [11].

[Word count: 47]

1.7 Cover Letter

A cover letter tailored to this advertisement is presented as a stand-alone letter in Appendix D, formatted so that it could be sent unaltered with a job application.

[Word count: 348 (see Appendix D)]

2 Data Set

2.1 Description of the Data Sets

Two public, account-free corpora of crowd-labelled user text underpin the analysis (Table 1). Davidson et al.’s hate speech corpus [3] holds 24,783 tweets, each judged by at least three CrowdFlower annotators and assigned one of three classes: 5.77% hate speech, 77.43% offensive language, 16.80% neither. Wulczyn et al.’s Wikipedia Talk corpus [13] holds

159,686 discussion comments carrying 1,598,289 toxicity judgements from 4,301 annotators, roughly ten per comment; 9.62% are toxic by majority vote. A companion file records each annotator’s demographics. Both download without an account from the URLs in Table 1.

[Word count: 90]

Table 1. The two public corpora used in this case study. Class balance is computed from the raw label files; the positive class is the item a moderator would be asked to act on.

	Dataset A: Davidson et al.	Dataset B: Wulczyn et al.
Source	Twitter	English Wikipedia talk pages
URL	https://github.com/t-davidson/hate-speech-and-offensive-language	https://figshare.com/articles/dataset/Wikipedia_Talk_Labels_Toxicity/4563973
Instances	24,783 tweets	159,686 comments
File size	2.4 MB (CSV)	109 MB (two TSV files)
Attributes	count, hate_speech, offensive_language, neither, class, tweet	rev_id, comment, year, logged_in, ns, sample, split
Annotations	≥ 3 per tweet (median 3, max 9)	1,598,289 judgements, 4,301 annotators, ≈10 per comment
Label scheme	3 classes: hate speech / offensive / neither	Binary toxicity per annotator, aggregated by majority
Positive class	Hate speech (5.77%, $n = 1,430$)	Toxic (9.62%, $n = 15,362$)
Mean length	85.4 characters	446.9 characters
Licence	Public research release (MIT)	CC0 1.0 Public Domain

2.2 Rationale for Selection

Both corpora are exactly the modality the advertised role’s classifiers consume: short, user-generated, crowd-labelled text. Their label schemes deliberately disagree — Davidson separates hate speech from merely offensive language, while Wikipedia collapses both into “toxic” — which lets me test whether a triage model survives a change of platform, the failure mode that matters in production. Limitations are real: both are English-only, Davidson’s lexicon-seeded sampling over-represents slurs, Wikipedia talk pages are not video comments, neither includes audio or vision, and majority-vote labels discard annotator disagreement.

[Word count: 83]

3 Data Analysis

3.1 Method

Both corpora were reduced to the same operational question a moderation queue actually asks: *should this item be escalated to a human reviewer?* In the Davidson corpus the positive class is hate speech; in the Wikipedia corpus it

is a comment that the majority of its annotators marked toxic. Text was represented as sublinear TF-IDF over word unigrams and bigrams, capped at 20,000 features.

Two algorithms were compared: a **linear support vector machine** [2], the standard strong baseline for high-dimensional sparse text [7], and a **feed-forward neural network** with one 64-unit hidden layer, which can represent feature interactions a linear boundary cannot. Neither was used in previous coursework. Both were implemented with scikit-learn [9] and evaluated under stratified five-fold cross validation. Critically, the vectoriser sits inside the pipeline and is therefore refitted on each training fold; fitting it once over the whole corpus would leak test-fold vocabulary into training and inflate every metric reported below [8].

3.2 Choice of Evaluation Metrics

Accuracy is the wrong headline for this case study, and Table 2 shows why concretely. On the Davidson corpus the neural network is the *more accurate* model (94.5% versus 92.2%) while being the substantially worse moderation tool: it recovers 13.5% of hate speech where the SVM recovers 41.1%. With a 5.77% positive class, a model that predicted “never escalate” would score 94.2% accuracy and be useless [8]. Four metrics were therefore chosen against the properties of this problem:

- **Precision and recall** on the positive class, because the two error types have different costs and different owners. A false negative leaves harmful content in front of users; a false positive removes a legitimate creator’s post and generates an appeal. Neither is captured by a single accuracy figure.
- **PR-AUC** as the primary threshold-independent score. Both corpora are dominated by negatives, the regime in which precision-recall curves are more informative than ROC curves, because ROC’s false-positive rate is diluted by a very large negative denominator and so looks flattering [4, 10].
- **ROC-AUC** reported alongside it, since it is insensitive to class distribution and therefore the fairer basis for comparing the same model across two corpora with different base rates [8].
- **Precision@1%**, the precision over the top 1% of the ranked stream. This is the metric the deployed system is actually judged on: moderator headcount is fixed, so what matters is the purity of the queue a reviewer works through, not performance at an arbitrary 0.5 threshold.

Model comparison uses a paired two-tailed *t*-test over the per-fold differences rather than a comparison of means, so that fold-to-fold variance is accounted for [8]. The test is reported with the caveat that repeated-CV folds are not fully independent, which makes it liberal [5].

3.3 What the Algorithms Reveal

The two models rank almost identically; they differ only in where they cut. This is the least obvious and most useful result. On both corpora the difference in PR-AUC is not statistically significant ($p = 0.299$ and $p = 0.778$), and on the Wikipedia corpus neither is ROC-AUC ($p = 0.651$). Yet precision and recall differ significantly every time. The two algorithms have learned an equivalent ordering of the data, and every apparent difference in Table 2 comes from the operating point rather than from model capacity — the SVM was given `class_weight="balanced"` and so cuts aggressively for recall, while the network, trained on the raw imbalance, cuts conservatively for precision. Figure 1 shows this directly: the curves sit on top of one another, and only the points chosen along them differ. For an engineer this reframes the decision. The choice between these two models is not a modelling question at all; it is a threshold policy set by how many moderators are rostered.

Table 2. Five-fold cross-validated performance, averaged across folds. Bold marks the better of the two models on each row. p -values are from paired two-tailed t -tests on per-fold differences. Note that on Dataset A the neural network is the more accurate model yet materially the worse moderation tool.

Metric	Dataset A: Davidson		Dataset B: Wikipedia	
	Linear SVM	Neural net	Linear SVM	Neural net
Accuracy	0.922	0.945	0.938	0.957
Precision	0.353	0.619	0.640	0.851
Recall	0.411	0.135	0.820	0.674
F1	0.379	0.216	0.719	0.752
ROC-AUC	0.829	0.801	0.959	0.960
PR-AUC	0.337	0.327	0.839	0.840
Precision@1%	0.632	0.640	1.000	0.999
<i>Paired t-test, SVM – neural net:</i>				
p (F1)		0.014*		<0.001*
p (PR-AUC)		0.299		0.778
p (ROC-AUC)		0.012*		0.651

* significant at $\alpha = 0.05$. Base rates: 5.77% (A), 9.62% (B).

The task is far harder on one corpus than the other. On Wikipedia both models are strong: PR-AUC ≈ 0.84 against a 9.62% base rate, and precision over the top 1% of the ranked stream is effectively perfect (1.000 and 0.999). A reviewer working the head of that queue would see almost nothing but genuine toxicity. On Davidson both models are weak: PR-AUC ≈ 0.33 , and precision@1% of only 0.63. The gap is not a data-volume artefact — it is that separating *hate speech* from *merely offensive language* is intrinsically harder than separating toxic from civil, which is precisely the problem Davidson et al. set out to demonstrate [3].

3.4 Are the Two Sources Complementary or Contradicting?

Both. They agree about the models and disagree about the labels, and separating those two claims is the substantive finding of this analysis.

They are **complementary about model selection**. Independently, on corpora that differ by a factor of six in size, by platform, by mean document length and by annotation protocol, both corpora return the same verdict: the two algorithms rank equivalently and the SVM achieves it at roughly one-fifth of the training cost. A conclusion that replicates across two unrelated sources is far stronger than one drawn from either alone.

They are **directly contradicting about what the label means**, and the contradiction is severe enough to invalidate any attempt to pool them. Training on Wikipedia and testing on Davidson, the SVM flags **73.3% of the entire stream** against a true base rate of 5.8%, and precision falls from 0.640 to 0.065 — ten times worse than the within-corpus figure. Figure 3 shows why. The Wikipedia-trained SVM flags 82.7% of Davidson’s hate speech and 82.7% of its *merely offensive*

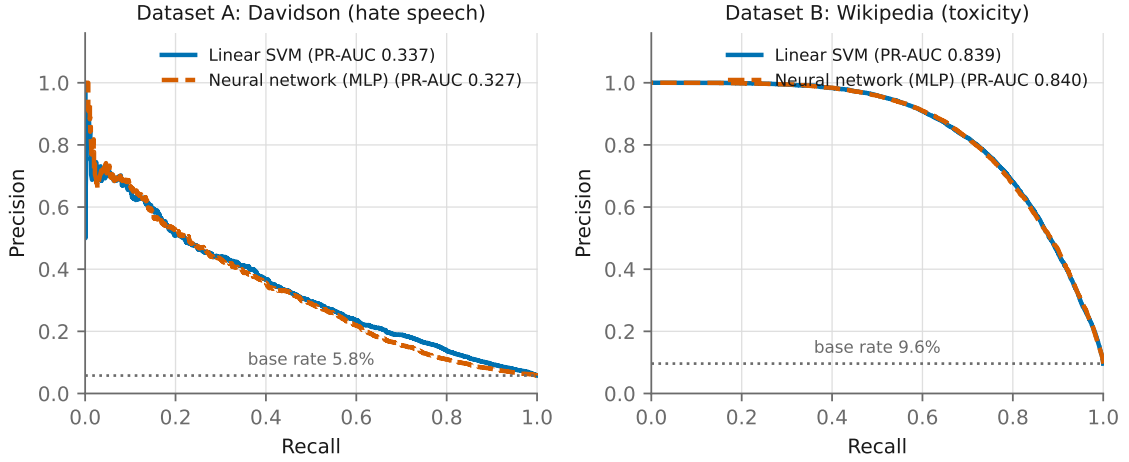


Fig. 1. Precision-recall curves, pooled across folds. The two algorithms are near-indistinguishable as *rankers* on both corpora; the significant differences in Table 2 come from threshold placement, not ranking quality.

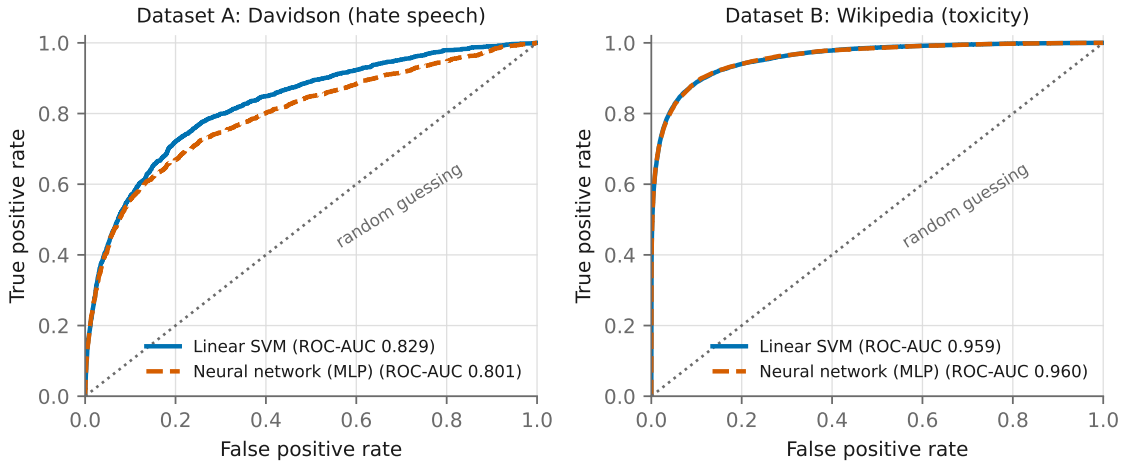


Fig. 2. ROC curves, pooled across folds. ROC-AUC of 0.96 on Dataset B looks excellent, but compare it with the PR-AUC of 0.84 in Figure 1: with 90% of the corpus negative, the false-positive rate is diluted and ROC flatters the models [4].

language — the identical rate. The model has no discriminative power whatsoever between the two classes, because the corpus it learned from never encoded a distinction between them. The neural network behaves the same way, flagging 68.4% of hate speech and 70.8% of non-hate offensive language, if anything inverting the ordering.

The reverse direction fails differently and just as informatively: a Davidson-trained neural network flags only 0.8% of Wikipedia against a 9.6% base rate, recovering 5.4% of genuine toxicity. Trained on a narrow, lexicon-seeded definition, it does not recognise most of what Wikipedia’s annotators consider harmful.

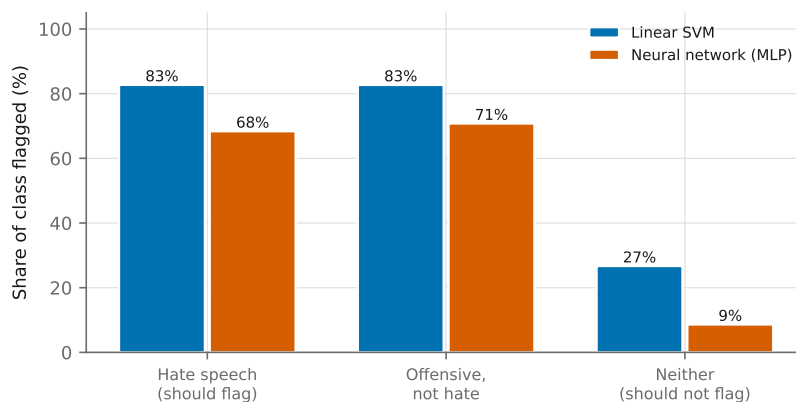


Fig. 3. Applying a Wikipedia-trained model to Davidson’s data. Both models flag genuine hate speech and merely offensive language at effectively the same rate — they cannot tell the two apart, because the Wikipedia label scheme never separated them. This is invisible to any single-corpus evaluation.

Do all data sources give insights? Not equally. Wikipedia supports a deployable triage model. Davidson does not — but its value is diagnostic rather than predictive: it is the only one of the two that can expose the over-flagging failure at all, precisely because it preserves the distinction the other corpus collapses. Discarding it as the “worse” dataset on PR-AUC would have discarded the finding.

3.5 Limitations and Implications for Practice

Three limitations bound these conclusions. Both corpora are English-only and text-only, whereas the advertised role concerns short-form video across many languages. Majority-vote aggregation discards annotator disagreement, and disagreement is signal — an item ten annotators split evenly on is a different object from one they unanimously cleared. And the paired *t*-test over five overlapping folds is liberal, so the significant results above should be read as indicative rather than conclusive [5].

A further concern is one the metrics here cannot see. Published work shows that classifiers trained on corpora of this kind systematically over-flag African-American English [1, 11], and the aggregate PR-AUC that looks healthy on Dataset B can conceal a much worse error rate for a specific community. The Wikipedia corpus ships annotator demographics for exactly this reason. Before deployment, the evaluation in Table 2 would need to be disaggregated by dialect and speaker group, not merely repeated at higher precision.

The practical recommendation follows from the first result. Ship the linear SVM: it ranks as well as the neural network on every threshold-independent measure, trains in roughly a fifth of the time, and its coefficients can be inspected when a creator appeals a removal. Then treat the decision threshold as an operational control tied to review capacity rather than a fixed model property. Most importantly, never validate such a model on a single labelled source. A system reporting 0.96 ROC-AUC in-corpus flagged three-quarters of a different platform’s traffic, and no amount of cross validation *within* one dataset would have revealed it [12].

– Content below this line does not count for the page/word limit. –

Generative AI Attribution and Declaration

AI Attribution Statement (generated with the AI Attribution Toolkit, <https://aiattribution.github.io/>):

This work was produced with the assistance of generative AI. AI was used to draft and edit prose, to write and debug the analysis code, and to structure the document. Dataset selection, the framing of the case study, the interpretation of results, and all final judgements are the author's own. All AI-assisted output was reviewed, tested against the data, and corrected by the author.

Specifically, Claude (Anthropic) was used to: locate and verify the public corpora and their access URLs; draft the Python analysis pipeline in Appendix C; draft and condense prose against the word limits in this specification; and format the $\text{\LaTeX}$ and $\text{\TeX}$ sources. Every numeric result reported in Section 3 was produced by executing the code in the linked repository, not by the language model. A record of the prompts, intermediate outputs and issues encountered is given in Appendix B.

Acknowledgments

I acknowledge the people of the Woi Wurrung and Boon Wurrung language groups of the eastern Kulin Nation, on whose unceded lands RMIT University's Melbourne campuses stand. I respectfully recognise their Ancestors and Elders, past and present.

I thank Thomas Davidson and colleagues, and Ellery Wulczyn and colleagues, for releasing the corpora that made this analysis possible.

References

- [1] Su Lin Blodgett, Lisa Green, and Brendan O'Connor. 2016. Demographic Dialectal Variation in Social Media: A Case Study of African-American English. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP'16)*. Association for Computational Linguistics, Austin, Texas, 1119–1130. doi:10.18653/v1/D16-1120
- [2] Corinna Cortes and Vladimir Vapnik. 1995. Support-Vector Networks. *Machine Learning* 20, 3 (1995), 273–297. doi:10.1007/BF00994018
- [3] Thomas Davidson, Dana Warmley, Michael Macy, and Ingmar Weber. 2017. Automated Hate Speech Detection and the Problem of Offensive Language. In *Proceedings of the 11th International AAAI Conference on Web and Social Media (ICWSM'17)*. AAAI Press, Montréal, Québec, Canada, 512–515. doi:10.1609/icwsml.v11i1.14955
- [4] Jesse Davis and Mark Goadrich. 2006. The Relationship Between Precision-Recall and ROC Curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML'06)*. ACM, Pittsburgh, Pennsylvania, USA, 233–240. doi:10.1145/1143844.1143874
- [5] Thomas G. Dietterich. 1998. Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms. *Neural Computation* 10, 7 (1998), 1895–1923. doi:10.1162/089976698300017197
- [6] eSafety Commissioner. 2022. Basic Online Safety Expectations. Australian Government. Retrieved 14 August 2026 from <https://www.esafety.gov.au/industry/basic-online-safety-expectations>
- [7] Thorsten Joachims. 1998. Text Categorization with Support Vector Machines: Learning with Many Relevant Features. In *Proceedings of the 10th European Conference on Machine Learning (ECML'98)*. Springer, Chemnitz, Germany, 137–142. doi:10.1007/BFb0026683
- [8] David Page. [n. d.]. Evaluating Machine Learning Methods. Lecture notes, CS760, University of Wisconsin–Madison. Retrieved 14 August 2026 from <http://pages.cs.wisc.edu/~dpage/cs760/evaluating.pdf>
- [9] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830. <https://www.jmlr.org/papers/v12/pedregosa11a.html>
- [10] Takaya Saito and Marc Rehmsmeier. 2015. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE* 10, 3 (2015), e0118432. doi:10.1371/journal.pone.0118432

- [11] Maarten Sap, Dallas Card, Saadia Gabriel, Yejin Choi, and Noah A. Smith. 2019. The Risk of Racial Bias in Hate Speech Detection. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL'19)*. Association for Computational Linguistics, Florence, Italy, 1668–1678. doi:10.18653/v1/P19-1163
- [12] Bertie Vidgen and Leon Derczynski. 2020. Directions in Abusive Language Training Data: A Systematic Review of Garbage In, Garbage Out. *PLOS ONE* 15, 12 (2020), e0243300. doi:10.1371/journal.pone.0243300
- [13] Ellery Wulczyn, Nithum Thain, and Lucas Dixon. 2017. Ex Machina: Personal Attacks Seen at Scale. In *Proceedings of the 26th International Conference on World Wide Web (WWW'17)*. International World Wide Web Conferences Steering Committee, Perth, Australia, 1391–1399. doi:10.1145/3038912.3052591

A Job Listing

The advertisement analysed in Section 1 is reproduced below as retrieved on 14 August 2026 from <https://au.gradconnection.com/employers/tiktok/jobs/tiktok-machine-learning-engineer-graduate-trust-and-safety-engineering-2027-start-5/>. Applications close 30 August 2026.

Machine Learning Engineer Graduate (Trust and Safety Engineering) – 2027 Start

Employer: TikTok | **Location:** Sydney, NSW, Australia | **Job type:** Graduate Jobs

Disciplines: Computer Science, Engineering Software, Information Systems | **Accepts international applicants:** Yes

Closing date: 30 August 2026, 11:59 pm

Source: <https://au.gradconnection.com/employers/tiktok/jobs/tiktok-machine-learning-engineer-graduate-trust-and-safety-engineering-2027-start-5/>

Retrieved: 14 August 2026. Reproduced for academic coursework (RMIT COSC2669/COSC2816, Individual Task 1).

Team Introduction

Building a world where people can safely discover, create and connect. The Trust & Safety (T&S) team at TikTok helps ensure that our global online community is safe and empowered to create and enjoy content across all of our applications. We have invested heavily in human and machine-based moderation to remove harmful content quickly and often before it reaches our general community.

We are looking for talented individuals to join our team in 2027. As a graduate, you will get opportunities to pursue bold ideas, tackle complex challenges, and unlock limitless growth. Launch your career where inspiration is infinite at our Company.

Successful candidates must be able to commit to an onboarding date by end of year 2027. Please state your availability and graduation date clearly in your resume.

As a Machine Learning Engineer, you'll have the chance to work with our clients and teams to address key business problems and identify areas of growth for the company. With your education and experience, you will be able to take on real-world challenges from day one.

Responsibilities

- Collaborate on research projects within the team or across countries
- Work with our world-class engineers to build industry-leading trust and safety systems for TikTok
- Develop and build up highly-scalable classifiers, tools, models and algorithms leveraging cutting-edge machine learning, computer vision and data mining technologies
- Improve our trust and safety strategy and work on model iterations
- Collaborate with cross-functional teams to protect TikTok globally
- Present research outcomes to internal and/or external audiences
- Work with engineering teams to apply technical achievements from our research to product prototyping
- Develop and implement innovative machine learning algorithms to manage business risks in TikTok's products and platforms
- Prototype and explore novel solutions, conduct experiments to validate hypotheses, and provide insights to Product and Tech teams

Minimum Qualifications

- Currently pursuing your PhD or Master degree in Computer Science or related engineering field
- Solid knowledge in at least one of the following areas: machine learning, pattern recognition, NLP, data mining, or computer vision
- Firm understanding of data structures and algorithms
- Great communication and teamwork skills

Preferred Qualifications

- Passion about techniques and solving challenging problems
- Previous experience in applications of machine learning, pattern recognition, NLP, data mining, or computer vision

By submitting an application for this role, you accept and agree to our global applicant privacy policy, which may be accessed here: <https://careers.tiktok.com/legal/privacy>

If you have any questions, please reach out to us at apac-earlycareers@tiktok.com

Fig. 4. Job advertisement: TikTok, Machine Learning Engineer Graduate (Trust and Safety Engineering) — 2027 Start, Sydney NSW.

B Development Process

This appendix documents how the executive summary was produced, as evidence of process rather than of outcome alone.

B.1 Pipeline

The analysis is five scripts, run in order, each writing artefacts the next consumes (Table 3). Every number quoted in Section 3 comes from `results/results.json`, which is regenerated from scratch by `03_analysis.py`.

Table 3. Analysis pipeline. Total runtime on a consumer laptop was approximately 25 minutes, dominated by the neural network folds on the larger corpus.

Step	Script	Purpose
1	<code>01_download_data.py</code>	Fetch both corpora from GitHub and Figshare; no account required
2	<code>02_profile_data.py</code>	Record exact row counts, attributes and class balance quoted in Section 2
3	<code>03_analysis.py</code>	Stratified 5-fold cross validation of both models on both corpora, cross-corpus transfer, paired <i>t</i> -tests
4	<code>04_job_ad_pdf.py</code>	Typeset the retrieved advertisement as Figure 4
5	<code>05_figures.py</code>	Render the precision-recall and ROC curves and the transfer chart

B.2 Core Modelling Code

Listing 1 shows the two estimators and the feature pipeline. The vectoriser sits *inside* the pipeline so that `StratifiedKFold` refits it on each training fold; fitting it once over the whole corpus would leak test-fold vocabulary and inflate every metric, the pitfall Page [8] raises on slide 35.

```

1 def make_vectoriser():
2     # Fitted inside every fold, never on the full corpus.
3     return TfidfVectorizer(
4         sublinear_tf=True, min_df=3, max_features=20000,
5         ngram_range=(1, 2), strip_accents="unicode", lowercase=True,
6     )
7
8 def make_models():
9     return {
10         "Linear SVM": Pipeline([
11             ("tfidf", make_vectoriser()),
12             ("clf", LinearSVC(C=1.0, class_weight="balanced",
13                             max_iter=5000, random_state=SEED)),
14         ]),
15         "Neural network (MLP)": Pipeline([
16             ("tfidf", make_vectoriser()),
17             ("clf", MLPClassifier(hidden_layer_sizes=(64,), activation="relu",
18                                 solver="adam", alpha=1e-4, batch_size=256,
19                                 learning_rate_init=1e-3, max_iter=30,

```

```

573 20         early_stopping=True, n_iter_no_change=3,
574 21         validation_fraction=0.1, random_state=SEED)),
575 22     ],
576 23 }
577 24
577 25 def precision_at_capacity(y_true, score, capacity=0.01):
578 26     """Precision over the top-scoring slice a moderation queue can review."""
579 27     k = max(1, int(round(len(y_true) * capacity)))
580 28     top = np.argsort(score)[: -1][ :k]
581 29     return float(y_true[top].mean()), k

```

Listing 1. Feature pipeline and the two estimators (code/03_analysis.py).

B.3 Issues Encountered and How They Were Addressed

- (1) **The first advertisement selected had closed.** The 2026-start version of this role expired before submission. Rather than cite a dead link, the employer’s job index was searched for the live equivalent; the 2027-start listing was confirmed open (an active *Apply* control, closing 30 August 2026) and substituted. The text of the two postings is materially identical.
- (2) **The obvious toxicity dataset was behind a login.** The widely used Jigsaw release is distributed through Kaggle, which requires an account and so fails the specification’s requirement that the link be accessible. The original Wulczyn et al. release on Figshare was used instead: same underlying annotations, CC0 licence, no account.
- (3) **Encoded whitespace polluted the features.** Wikipedia comments store line and tab breaks as the literal strings `NEWLINE_TOKEN` and `TAB_TOKEN`. Left in place these become two of the highest-frequency terms in the vocabulary and carry no signal. They are stripped before vectorising.
- (4) **The two estimators expose different score interfaces.** `LinearSVC` provides `decision_function` but not `predict_proba`. Because both ROC and precision-recall curves depend only on the *ranking* induced by the score, the raw decision values are used directly; no calibration step is needed for the curves, though it would be needed before interpreting a score as a probability in production.
- (5) **Neural network runtime.** A dense hidden layer over a full-vocabulary sparse matrix was too slow to cross-validate on the larger corpus. The vocabulary was capped at 20,000 features and the network reduced to a single 64-unit hidden layer with early stopping. Both models share the identical vectoriser configuration so the comparison stays fair.
- (6) **Class imbalance made the default threshold useless for the SVM.** At the default decision boundary the minority class was barely predicted at all. `class_weight="balanced"` was enabled, which is why the SVM’s operating point favours recall over precision — a choice revisited in Section 3.

B.4 Use of Generative AI

Claude (Anthropic) was used throughout, in the manner declared on page 3. Representative prompts:

- “Find live machine learning engineer job advertisements in Australia and verify the listing URLs actually resolve.”
- “Propose two publicly available datasets relevant to content-moderation triage that download without an account, and confirm the download URLs respond.”
- “Write a stratified cross-validation harness comparing a linear SVM and an MLP, fitting the vectoriser inside each fold, and report precision, recall, F1, ROC-AUC and PR-AUC plus a paired *t*-test across folds.”

- “Condense this paragraph to at most 50 words without losing the citation.”

Two defects survived into a compiled draft and were caught only by rendering the PDF and inspecting every page rather than trusting the source. The typeset job advertisement originally ran to two pages, of which Figure 4 included only the first — the preferred qualifications were silently missing. Separately, the cover letter inherited the ACM template’s one-and-a-half line spacing and broke across two pages, defeating the purpose of a stand-alone letter. Both are corrected in this version. The lesson generalises to the analysis itself: generated output looks finished long before it is correct, and the only reliable check is to inspect the artefact rather than the code that produced it.

C Link to GitHub Repository

All code, the downloaded-data manifest and the generated results are available at:

<https://github.com/heetchanchad15/Case-Studies-Assignment>

677 **D Cover Letter**

678 The letter below is written to stand alone: it carries its own sender and recipient blocks and no report formatting, so it
679 could be sent unaltered with an application. **[Word count: 348]**
680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

Heet Chanchad

Melbourne VIC, Australia
s4218211@student.rmit.edu.au

14 August 2026

Early Careers Team
Trust & Safety Engineering
TikTok Australia
Sydney NSW

Re: Machine Learning Engineer Graduate (Trust and Safety Engineering) — 2027 Start

Dear Hiring Team,

I am writing to apply for the Machine Learning Engineer Graduate role in Trust and Safety Engineering. I am completing a Master of Data Science at RMIT University's City campus, which I began in March 2026 and will finish in March 2028, and I am available to begin from that date. I want to spend my career building systems that reduce harm at scale, not systems that maximise attention, and your team is one of the few places where that is the explicit brief.

The advertisement asks for someone who can “develop and build up highly-scalable classifiers” and “work on model iterations”. My most recent project is that problem in miniature. I built a content-triage pipeline over two public corpora — 24,783 crowd-labelled tweets and 159,686 Wikipedia Talk comments carrying 1.6 million toxicity judgements — and compared a linear support vector machine against a feed-forward neural network under stratified five-fold cross validation.

Two findings from that work speak directly to your team's problem. First, I evaluated the models the way a moderation queue actually runs. Because only 5.8% and 9.6% of the two corpora are removal-worthy, accuracy is close to meaningless, so I reported precision, recall and area under the precision-recall curve, and measured precision across the top 1% of the ranked stream — the slice a fixed review team could genuinely process in a day. Second, I tested whether a model survives a change of source. A classifier scoring 0.96 ROC-AUC on its own corpus flagged 73% of the other platform's traffic against a true base rate of 5.8%, because the two label schemes disagree about whether offensive language is the same thing as hate speech. That failure is invisible in single-dataset metrics and would surface in production as mass wrongful removals of ordinary speech.

I also treat fairness as part of the engineering rather than a review step after it. Published work shows hate-speech classifiers systematically over-flag African-American English, and the Wikipedia corpus ships annotator demographics precisely because who labels the data decides what counts as toxic. Before I would ship a classifier, I would want its error rates broken down by dialect and community, not just its headline score.

I would welcome the opportunity to discuss how I could contribute to the team. Thank you for your consideration.

Yours sincerely,

Heet Chanchad